



UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

CARRERA:

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

NRC: 30693

ASIGNATURA:

METODOLOGÍA DE DESARROLLO DE SOFTWARE

INTEGRANTES:

CAYO JANINE

CHAFLA SANTIAGO

GUAYCHA ANTHONY

MADRID EDMIR

DOCENTE:

ING. JENNY RUIZ, MGT.

REQUISITOS FUNCIONALES

Código	Requisito funcional	Actor	Entrada	Salida
REQ010	Visualizar catálogo	Cliente	Solicitud de catálogo (sesión activa)	Lista de productos con stock > 0
REQ011	Agregar al carrito	Cliente	Producto seleccionado y cantidad	Carrito actualizado
REQ012	Registrar pedido	Cliente	Confirmación del carrito	Pedido almacenado
REQ013	Ver historial de pedidos	Cliente	Solicitud de historial (idCliente de sesión)	Lista de pedidos anteriores

DIAGRAMA DE CASOS DE USO

@startuml
left to right direction
actor Cliente

```
rectangle "Sistema MPG-CAPS" {  
    usecase "Iniciar sesión" as UC1  
    usecase "Registrar cliente" as UC6  
    usecase "Visualizar catálogo de productos" as UC10  
    usecase "Agregar productos al carrito" as UC11  
    usecase "Registrar pedido" as UC12  
    usecase "Ver historial de mis pedidos" as UC13  
}
```

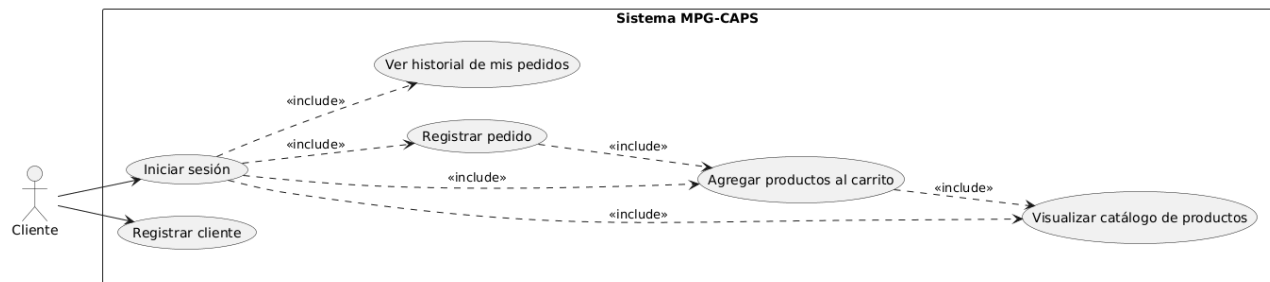
Cliente --> UC1
Cliente --> UC6

UC1 ..> UC10 : <<include>>
UC1 ..> UC11 : <<include>>
UC1 ..> UC12 : <<include>>
UC1 ..> UC13 : <<include>>

UC11 ..> UC10 : <<include>>
UC12 ..> UC11 : <<include>>

@enduml

Imagen del Diagrama



Interpretación

El diagrama de casos de uso representa las funcionalidades del módulo de compras del sistema MPG-CAPS correspondientes al tercer sprint, orientadas exclusivamente al actor Cliente.

Para acceder a cualquiera de sus opciones (catálogo, carrito, pedidos e historial), el Cliente debe iniciar sesión previamente; por ello, el caso de uso "Iniciar sesión" incluye a los cuatro casos de uso propios del cliente (visualizar catálogo, agregar al carrito, registrar pedido y ver historial). Si el Cliente aún no posee una cuenta, dispone del caso de uso "Registrar cliente" para crearla antes de poder loguearse.

Adicionalmente, se mantienen las dependencias funcionales entre los casos de uso propios del sprint: agregar productos al carrito incluye la consulta previa del catálogo, y registrar un pedido incluye contar con productos ya agregados al carrito.

4. ESPECIFICACIÓN BREVE DE CASOS DE USO

Caso de uso	Actor	Precondición	Flujo principal resumido	Resultado esperado
Visualizar catálogo de productos	Cliente	El cliente debe estar logueado	Acceder al catálogo → consultar productos con stock > 0 en MongoDB → mostrar lista	Catálogo visualizado
Agregar productos al carrito	Cliente	El cliente debe estar logueado y debe existir al menos un producto con stock disponible	Seleccionar producto → ingresar cantidad → validar cantidad ≤ stock → agregar al carrito temporal	Carrito actualizado
Registrar pedido	Cliente	El cliente debe estar logueado y el carrito debe contener al menos un producto	Confirmar carrito → calcular total → guardar pedido en MongoDB (idCliente, fecha, items, total) → limpiar carrito	Pedido registrado
Ver historial de mis pedidos	Cliente	El cliente debe estar logueado y tener pedidos previos registrados	Solicitar historial → consultar pedidos por idCliente → mostrar lista ordenada por fecha descendente	Historial mostrado

5. PUENTE HACIA CLASES DE ANÁLISIS (Boundary-Control-Entity)

@startuml

left to right direction

actor Cliente

boundary Vista_Catalogo

boundary PanelProducto

boundary Vista_Previsualizar

boundary Vista_Historial

control Controlador_Catalogo

control Controlador_PanelProducto

control Controlador_Previsualizar

control Controlador_Historial

entity Producto

entity Factura

entity DetalleFactura

entity Repositorio_Producto

entity Repositorio_Clientes

entity Repositorio_Facturas

entity Crear_PDF

entity Repositorio_Reporte_Facturas

Cliente --> Vista_Catalogo

Vista_Catalogo --> Controlador_Catalogo

Controlador_Catalogo --> Repositorio_Producto

Controlador_Catalogo --> Repositorio_Clientes

Controlador_Catalogo --> Factura

Controlador_Catalogo --> PanelProducto

PanelProducto --> Controlador_PanelProducto

Controlador_PanelProducto --> Producto

Controlador_PanelProducto --> Factura

Controlador_PanelProducto --> DetalleFactura

Controlador_Catalogo --> Vista_Previsualizar

Vista_Previsualizar --> Controlador_Previsualizar

Controlador_Previsualizar --> Repositorio_Facturas

Controlador_Previsualizar --> Crear_PDF

Controlador_Previsualizar --> Repositorio_Producto

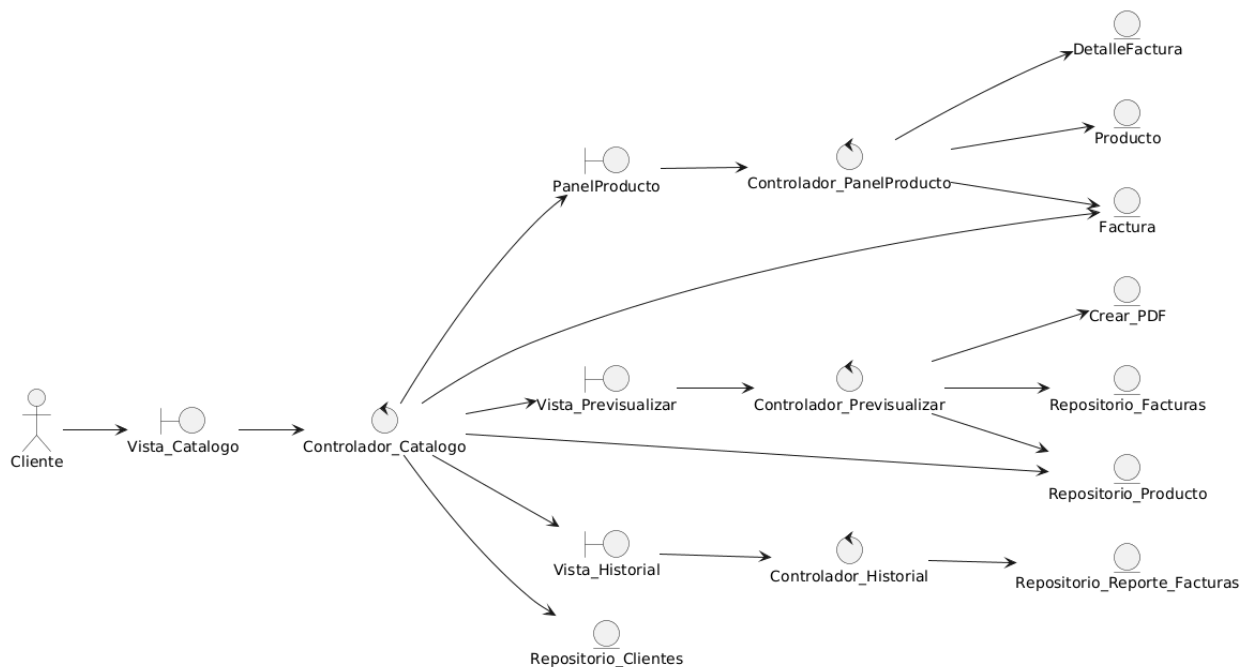
Controlador_Catalogo --> Vista_Historial

Vista_Historial --> Controlador_Historial

Controlador_Historial --> Repositorio_Reporte_Facturas

@enduml

Imagen del Diagrama



Interpretación

El diagrama Boundary-Control-Entity representa el flujo de interacción del Cliente con el módulo de compras del sistema MPG-CAPS, correspondiente al tercer sprint.

El Cliente accede a Vista_Catalogo, la cual delega en Controlador_Catalogo la construcción del catálogo (crearCatalogo) a partir de Repositorio_Producto, así como la obtención de los datos del cliente mediante Repositorio_Clientes, ambos asociados al objeto Factura que se mantiene durante toda la sesión de compra.

Cada producto mostrado en el catálogo se despliega mediante el boundary PanelProducto, el cual delega en Controlador_PanelProducto el método agregar(), encargado de validar el stock, crear un DetalleFactura y añadirlo a la Factura.

Al confirmar la compra, Controlador_Catalogo invoca comprar(), abriendo el boundary Vista_Previsualizar, cuyo control Controlador_Previsualizar coordina la generación del PDF (Crear_PDF), el registro de la factura (Repositorio_Facturas) y la actualización de stock (Repositorio_Producto).

Finalmente, el Cliente puede consultar Vista_Historial, gestionada por Controlador_Historial, que obtiene los reportes desde Repositorio_Reporte_Facturas.

6. DIAGRAMA DE CLASES

@startuml

```

class Controlador_Catalogo {
    - factura : Factura
    - repositorio_producto : Repositorio_Producto
    + crearCatalogo(lista_producto : ArrayList<Producto>)
    + comprar()
}
  
```

```

class Controlador_PanelProducto {
    - factura : Factura
    - producto : Producto
}
  
```

```
+ agregar()  
}
```

```
class Controlador_Previsualizar {  
- factura : Factura  
+ generarFactura()  
}
```

```
class Controlador_Historial {  
+ verDetalle()  
}
```

```
class Factura {  
- idFactura : String  
- detalle : ArrayList<DetalleFactura>  
+ agregarDetalle(DetalleFactura)  
+ calcularSubtotal()  
+ precioTotal()  
}
```

```
class DetalleFactura {  
- nombreProducto : String  
- cantidad : int  
- precioUnitario : double  
+ aumentarCantidad(int)  
+ calcularTotalUnitario()  
}
```

```
class Producto {  
- numeroSerie : String  
- nombre : String  
- cantidad : int  
- precioUnitario : double  
+ reducirStock(int reduccion)  
}
```

```
class Repositorio_Producto {  
+ getLista() : ArrayList<Producto>  
+ buscar_reducirStock(detalle : ArrayList<DetalleFactura>)  
}
```

```
class Repositorio_Facturas {  
+ generarCodigoFactura()  
+ agregarDetalles()  
}
```

```
class Crear_PDF {  
+ generarPDF(factura : Factura) : String  
}
```

Controlador_Catalogo --> Repositorio_Producto

Controlador_Catalogo --> Factura

Controlador_PanelProducto --> Producto

Controlador_PanelProducto --> Factura

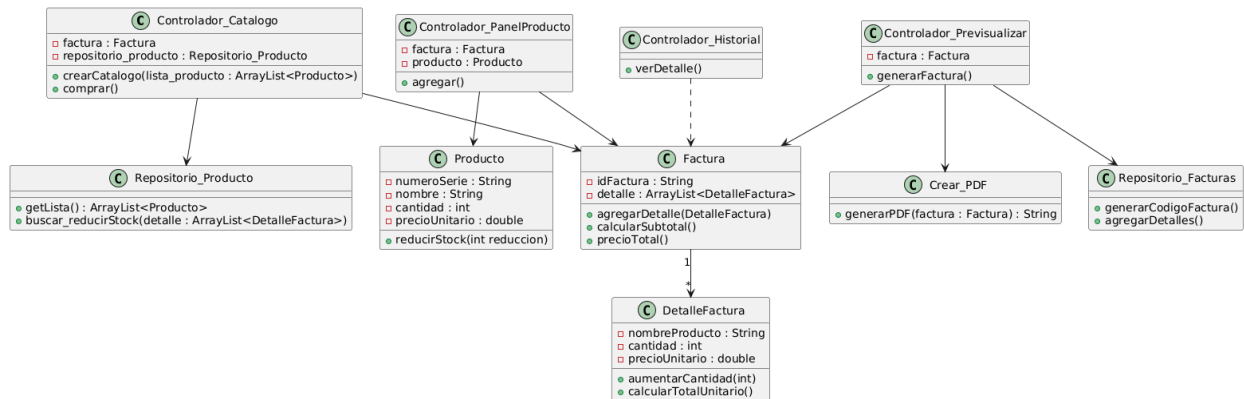
Controlador_Previsualizar --> Factura

Controlador_Previsualizar --> Repositorio_Facturas
Controlador_Previsualizar --> Crear_PDF
Controlador_Historial ..> Factura

Factura "1" --> "*" DetalleFactura

@enduml

Imagen del Diagrama



Interpretación

El diagrama de clases simplificado muestra únicamente los elementos esenciales del proceso de compra del tercer sprint: la construcción del catálogo, la adición de productos al carrito, la confirmación del pedido y la consulta del historial.

Controlador_Catalogo arma el catálogo a partir de Repositorio_Producto y coordina la confirmación de la compra. Controlador_PanelProducto valida el stock del Producto seleccionado y agrega o actualiza un DetalleFactura dentro de la Factura activa.

Controlador_Previsualizar cierra el ciclo generando el comprobante en PDF (Crear_PDF) y registrando la venta en Repositorio_Facturas. Controlador_Historial permite consultar las facturas ya registradas.

Se omiten deliberadamente las clases de soporte secundario (Datos_Cliente,

Repositorio_Clientes, ServicioEmail, Repositorio_Reporte_Facturas) y los métodos de acceso (getters/setters) para mantener el foco en la lógica de negocio del sprint.

La clase Repositorio_Producto se encarga de administrar la información almacenada en MongoDB mediante operaciones de búsqueda, edición, eliminación y actualización de los productos registrados.